

Hands-On AI with Python

Level 1: Python Foundations for AI

Complete Guide with Explanations & Runnable Sample Code
Author: Venkatesan Vetrimurasu | YouTube: Venkat's Tech Lab

This guide covers every essential Python skill you need before starting Machine Learning and Deep Learning. Each topic contains: why it matters for AI, key concepts, a complete runnable sample, and a practice tip.

How to use: Create a folder `level1_python_foundations`, copy each sample into its own `.py` file, run it, then modify and break it. That is how real learning happens.

Table of Contents

1. Variables, types, f-strings — Unit Converter
2. Lists, dicts, sets, tuples — Word Counter
3. Loops and comprehensions — Filter a Dataset
4. Functions, *args, **kwargs, lambdas — Mini Calculator
5. File I/O (CSV, JSON, TXT) + pathlib — Contact Manager
6. Error handling and custom exceptions
7. OOP and dataclasses — Dataset class
8. Modules, packages, venv, pip / uv
9. Reproducibility: seeds, pinned environments, uv
10. Type hints with mypy
11. Pydantic: validation, schemas, settings
12. Generators, iterators, decorators
13. Context managers (with statement) ← NEW
14. collections module (defaultdict, deque, NamedTuple) ← NEW
15. requests and REST APIs
16. Async with asyncio and httpx
17. Simple CLI with argparse ← NEW

1. Variables, Types, f-strings — Unit Converter

Why this matters for AI

You constantly convert units (pixels ↔ meters, temperatures, token counts, learning rates). f-strings are the cleanest way to log training progress and create readable reports.

Key concepts

- Variables hold data. Python is dynamically typed but values have types.
- f-strings (f"...") embed expressions directly inside strings.
- Type conversion: int(), float(), str(), bool().

Complete Sample

```
# unit_converter.py
def convert_length(value: float, from_unit: str, to_unit: str) -> float:
    to_meters = {
        "m": 1.0, "km": 1000.0, "cm": 0.01, "mm": 0.001,
        "mile": 1609.34, "yard": 0.9144, "foot": 0.3048, "inch": 0.0254,
    }
    if from_unit not in to_meters or to_unit not in to_meters:
        raise ValueError(f"Unsupported unit. Supported: {list(to_meters.keys())}")
    meters = value * to_meters[from_unit]
    return meters / to_meters[to_unit]

value, from_u, to_u = 5.0, "km", "mile"
converted = convert_length(value, from_u, to_u)
print(f"{value} {from_u} = {converted:.4f} {to_u}")
# Output: 5.0 km = 3.1069 mile
```

Practice tip: Extend to temperature ($^{\circ}\text{C} \leftrightarrow ^{\circ}\text{F} \leftrightarrow \text{K}$) and weight. Add a simple CLI using `input()`.

2. Lists, Dicts, Sets, Tuples — Word Counter

Why this matters for AI

Almost every dataset is a list of records. Feature dictionaries, vocabulary sets, and immutable configuration tuples appear constantly. Counter is the fastest way to build frequency tables.

Key concepts

- list — ordered, mutable | dict — key→value | set — unique elements | tuple — immutable
- collections.Counter — specialized dict for counting

Complete Sample

```
# word_counter.py
from collections import Counter

text = """Python is great. Python is powerful.
Learning Python with hands-on projects is the best way."""

words = text.lower().replace(".", "").replace(",", "").split()
word_counts = Counter(words)

print("Word frequencies:")
for word, count in word_counts.most_common():
    print(f"{word:15} → {count}")

unique_words = set(words)
print(f"\nTotal unique words: {len(unique_words)}")
print(f"Unique words: {sorted(unique_words)}")
```

Practice tip: Count bigrams (pairs of consecutive words). This is the foundation of simple language models.

3. Loops and Comprehensions — Filter a Dataset

Why this matters for AI

Data cleaning and feature extraction are mostly filtering and transforming collections. Comprehensions are faster to write and often faster to run than explicit loops.

Complete Sample

```
# filter_dataset.py
students = [
    {"name": "Alice", "age": 22, "score": 88},
    {"name": "Bob", "age": 19, "score": 72},
    {"name": "Charlie", "age": 25, "score": 95},
    {"name": "Diana", "age": 20, "score": 61},
    {"name": "Eve", "age": 23, "score": 79},
]

passed = [s["name"] for s in students if s["score"] >= 75]
print("Passed:", passed)

high_scores = {s["name"]: s["score"] for s in students if s["score"] >= 80}
print("High scores:", high_scores)

older = [s for s in students if s["age"] > 21]
avg = sum(s["score"] for s in older) / len(older)
print(f"Average score (age > 21): {avg:.1f}")
```

Practice tip: Rewrite the average using `statistics.mean` and a generator expression.

4. Functions, *args, **kwargs, Lambdas — Mini Calculator

Why this matters for AI

Every training loop, metric calculator, and data transform is a function. *args / **kwargs let you write flexible APIs (exactly how PyTorch and scikit-learn do it).

Complete Sample

```
# calculator.py
def calculate(operation, *args, **kwargs):
    if not args:
        raise ValueError("At least one number required")
    precision = kwargs.get("precision", 2)
    verbose = kwargs.get("verbose", False)

    if operation == "add":
        result = sum(args)
    elif operation == "multiply":
        result = 1
        for num in args:
            result *= num
    elif operation == "average":
        result = sum(args) / len(args)
    else:
        raise ValueError(f"Unknown operation: {operation}")

    result = round(result, precision)
    if verbose:
        print(f"Operation: {operation} | Numbers: {args} → Result: {result}")
    return result

square = lambda x: x ** 2
is_even = lambda x: x % 2 == 0

print(calculate("add", 10, 20, 30, precision=0, verbose=True))
print(calculate("multiply", 2, 3, 4, 5))
print(square(7))
print(list(filter(is_even, range(10))))
```

Practice tip: Add a 'power' operation. Write a function that accepts any number of lists and returns element-wise averages.

5. File I/O (CSV, JSON, TXT) + pathlib — Contact Manager

Why this matters for AI

Every real project starts with loading data from disk. JSON is standard for configs and API responses. CSV is the everyday format for tabular datasets. pathlib is the modern way to handle paths.

Complete Sample

```
# contact_manager.py
import json, csv
from pathlib import Path

CONTACTS_FILE = Path("contacts.json")

def load_contacts():
    if CONTACTS_FILE.exists():
        with open(CONTACTS_FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    return []

def save_contacts(contacts):
    with open(CONTACTS_FILE, "w", encoding="utf-8") as f:
        json.dump(contacts, f, indent=2)

def add_contact(name: str, phone: str, email: str):
    contacts = load_contacts()
    contacts.append({"name": name, "phone": phone, "email": email})
    save_contacts(contacts)
    print(f"Added: {name}")

def export_to_csv(filename="contacts.csv"):
    contacts = load_contacts()
    with open(filename, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["name", "phone", "email"])
        writer.writeheader()
        writer.writerows(contacts)
    print(f"Exported to {filename}")

def show_contacts():
    contacts = load_contacts()
    if not contacts:
        print("No contacts yet.")
        return
    for i, c in enumerate(contacts, 1):
        print(f"{i}. {c['name']} | {c['phone']} | {c['email']}")

add_contact("Alice", "123-456-7890", "alice@email.com")
add_contact("Bob", "987-654-3210", "bob@email.com")
show_contacts()
export_to_csv()
```

Practice tip: Add search-by-name and delete functions. Store training-run metadata the same way.

6. Error Handling and Custom Exceptions

Why this matters for AI

Data files are messy. Models fail. APIs time out. Robust pipelines catch specific errors, log them, and continue or fail gracefully.

Complete Sample

```
# safe_reader.py
class FileEmptyError(Exception):
    """Raised when a file is empty."""
    pass

class InvalidFormatError(Exception):
    """Raised when file content is not in expected format."""
```

```
pass

def safe_read_numbers(filepath: str) -> list[float]:
    try:
        with open(filepath, "r") as f:
            content = f.read().strip()
        if not content:
            raise FileNotFoundError(f"File '{filepath}' is empty")
        numbers = []
        for line in content.splitlines():
            line = line.strip()
            if not line:
                continue
            try:
                numbers.append(float(line))
            except ValueError:
                raise InvalidFormatError(f"Invalid number found: '{line}'")
        return numbers
    except FileNotFoundError:
        print(f"Error: File '{filepath}' does not exist.")
        return []
    except (FileNotFoundError, InvalidFormatError) as e:
        print(f"Custom Error: {e}")
        return []
    except Exception as e:
        print(f"Unexpected error: {e}")
        return []

with open("numbers.txt", "w") as f:
    f.write("10\n20\n3.5\nabc\n40\n")
print(safe_read_numbers("numbers.txt"))
print(safe_read_numbers("nonexistent.txt"))
```

Practice tip: Write a function that loads a JSON config and raises a custom `ConfigError` if required keys are missing.

7. OOP and Dataclasses — Dataset Class

Why this matters for AI

PyTorch Dataset, scikit-learn estimators, and almost every modern library are built with classes. Dataclasses remove boilerplate and make data containers clean.

Complete Sample

```
# dataset_class.py
from dataclasses import dataclass, field
from typing import List, Any
import statistics

@dataclass
class Sample:
    features: List[float]
    label: Any = None

@dataclass
class Dataset:
    name: str
    samples: List[Sample] = field(default_factory=list)

    def add_sample(self, features: List[float], label=None):
        self.samples.append(Sample(features, label))

    def __len__(self):
        return len(self.samples)

    def mean_feature(self, index: int) -> float:
        values = [s.features[index] for s in self.samples]
        return statistics.mean(values)

    def summary(self):
```

```
print(f"Dataset: {self.name}")
print(f"Number of samples: {len(self)}")
if self.samples:
    n_features = len(self.samples[0].features)
    print(f"Features per sample: {n_features}")
    for i in range(n_features):
        print(f"  Feature {i} mean: {self.mean_feature(i):.2f}")

ds = Dataset(name="Iris-like")
ds.add_sample([5.1, 3.5, 1.4, 0.2], label="setosa")
ds.add_sample([7.0, 3.2, 4.7, 1.4], label="versicolor")
ds.add_sample([6.3, 3.3, 6.0, 2.5], label="virginica")
ds.summary()
print(f"Length: {len(ds)}")
```

Practice tip: Add a `train_test_split(self, ratio=0.8)` method that returns two new `Dataset` objects.

8. Modules, Packages, venv, pip / uv

Why this matters for AI

You will install dozens of libraries. Virtual environments keep projects isolated. Proper packaging lets you reuse your own utility code across experiments.

Commands + Sample Package

```
# Terminal
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install --upgrade pip
pip install requests pandas
pip freeze > requirements.txt

mkdir -p myutils
# myutils/math_helpers.py
def clamp(value: float, min_val: float, max_val: float) -> float:
    return max(min_val, min(value, max_val))

# myutils/__init__.py
from .math_helpers import clamp
__all__ = ["clamp"]

# main.py
from myutils import clamp
print(clamp(15, 0, 10)) # → 10
```

Practice tip: Create a package 'aiutils' with a seed-setting function and a timer decorator. Install with `pip install -e .`

9. Reproducibility: Seeds, Pinned Environments, uv

Why this matters for AI

If you cannot reproduce a result, you cannot trust it. Setting random seeds and pinning exact package versions are the two most important habits for serious work. uv is a modern, extremely fast replacement for pip + venv.

Complete Sample

```
# reproducible.py
import random
import numpy as np

def set_seed(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    # Later for PyTorch:
    # import torch
    # torch.manual_seed(seed)
    # if torch.cuda.is_available():
    #     torch.cuda.manual_seed_all(seed)

set_seed(42)
print(random.random())
print(np.random.rand(3))

# Using uv (recommended)
# curl -LSf https://astral.sh/uv/install.sh | sh
# uv venv && source .venv/bin/activate
# uv pip install numpy pandas
# uv pip freeze > requirements.txt
```

Practice tip: Extend `set_seed` to also set `PYTHONHASHSEED` and `torch.backends.cudnn.deterministic = True`.

10. Type Hints with mypy

Why this matters for AI

Type hints document your code and catch entire classes of bugs before runtime. Libraries like PyTorch, Pydantic, and FastAPI rely heavily on them.

Complete Sample

```
# typed_example.py
from typing import List, Dict, Optional, Union

def average(numbers: List[float]) -> float:
    if not numbers:
        raise ValueError("List cannot be empty")
    return sum(numbers) / len(numbers)

def find_user(users: Dict[str, dict], user_id: str) -> Optional[dict]:
    return users.get(user_id)

def process(value: Union[int, str]) -> str:
    return str(value).upper()

# pip install mypy
# mypy typed_example.py
```

Practice tip: Add type hints to the Dataset class from topic 7, then run mypy and fix any issues.

11. Pydantic: Validation, Schemas, Settings

Why this matters for AI

Pydantic is the de-facto standard for data validation in modern Python. It powers FastAPI, LangChain, many LLM tools, and configuration systems.

Complete Sample

```
# pydantic_demo.py
from pydantic import BaseModel, Field, EmailStr, field_validator
from pydantic_settings import BaseSettings

class User(BaseModel):
    id: int
    name: str = Field(..., min_length=2, max_length=50)
    email: EmailStr
    age: int = Field(..., ge=0, le=120)

    @field_validator("name")
    @classmethod
    def name_must_be_capitalized(cls, v: str) -> str:
        if not v[0].isupper():
            raise ValueError("Name must start with capital letter")
        return v

class Settings(BaseSettings):
    api_key: str
    debug: bool = False
    max_tokens: int = 1000
    class Config:
        env_file = ".env"

user = User(id=1, name="Alice", email="alice@example.com", age=28)
print(user.model_dump())
# pip install pydantic pydantic-settings email-validator
```

Practice tip: Create a `TrainingConfig` model with `learning_rate`, `batch_size`, `epochs`, `model_name`. Load from `.env` or `JSON`.

12. Generators, Iterators, Decorators

Why this matters for AI

Generators let you process huge datasets without loading everything into memory. Decorators add timing, logging, retry logic, or caching without modifying core code.

Complete Sample

```
# generators_decorators.py
import time
from functools import wraps

def countdown(n: int):
    while n > 0:
        yield n
        n -= 1
print(list(countdown(5)))

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timer
```

```
def slow_function():
    time.sleep(0.5)
    return "Done"
print(slow_function())
```

Practice tip: Write a retry decorator (max 3 attempts). Write a generator that yields batches of a list.

13. Context Managers (with statement) ← NEW

Why this matters for AI

Context managers guarantee cleanup (closing files, releasing GPU memory, unlocking resources) even when exceptions occur. You already use them with `open()`; now you will create your own.

Key concepts

- with statement calls `__enter__` and `__exit__` automatically
- `contextlib.contextmanager` decorator for generator-based managers
- Critical for files, locks, database connections, temporary directories, GPU contexts

Complete Sample

```
# context_managers.py
from contextlib import contextmanager
import time

# 1. Class-based context manager
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"Elapsed: {self.elapsed:.4f}s")
        return False # do not suppress exceptions

with Timer() as t:
    time.sleep(0.3)
    print("Working...")

# 2. Generator-based context manager
@contextmanager
def open_and_count(path: str):
    f = open(path, "r", encoding="utf-8")
    try:
        lines = f.readlines()
        yield lines
    finally:
        f.close()
        print(f"Closed {path} ({len(lines)} lines)")

with open("numbers.txt", "w") as f:
    f.write("10\n20\n30\n")

with open_and_count("numbers.txt") as lines:
    print("Content:", [l.strip() for l in lines])
```

Practice tip: Write a context manager that temporarily changes the working directory and restores it afterward.

14. collections Module (defaultdict, deque, namedtuple) ← NEW

Why this matters for AI

defaultdict removes repetitive key-existence checks. deque is ideal for sliding windows and queues. namedtuple gives you lightweight immutable records. All three appear constantly in data pipelines and simple models.

Complete Sample

```
# collections_demo.py
from collections import defaultdict, deque, namedtuple, Counter

# 1. defaultdict – group items without KeyError
grouped = defaultdict(list)
pairs = [("fruit", "apple"), ("fruit", "banana"), ("veg", "carrot"), ("fruit", "orange")]
for category, item in pairs:
    grouped[category].append(item)
print(dict(grouped))
# {'fruit': ['apple', 'banana', 'orange'], 'veg': ['carrot']}
```

```
# 2. deque – efficient sliding window / queue
window = deque(maxlen=3)
for x in [10, 20, 30, 40, 50]:
    window.append(x)
    print(list(window))
# [10] → [10,20] → [10,20,30] → [20,30,40] → [30,40,50]
```

```
# 3. namedtuple – lightweight immutable record
Sample = namedtuple("Sample", ["features", "label"])
s = Sample(features=[1.2, 3.4], label="positive")
print(s.features, s.label)
print(s._asdict())
```

```
# 4. Counter (already used in word counter)
print(Counter("abracadabra").most_common(3))
```

Practice tip: Implement a simple moving-average calculator using `deque(maxlen=n)`. Use `defaultdict(int)` to count co-occurrences of word pairs.

15. requests and REST APIs

Why this matters for AI

You will download datasets, call model APIs (OpenAI, Anthropic, Hugging Face), and fetch web data. Understanding status codes, headers, and JSON responses is essential.

Complete Sample

```
# api_demo.py
import requests

def get_github_user(username: str) -> dict:
    url = f"https://api.github.com/users/{username}"
    response = requests.get(url, timeout=10)
    if response.status_code == 200:
        return response.json()
    print(f"Error {response.status_code}: {response.text}")
    return {}

user = get_github_user("octocat")
if user:
    print(f"Name: {user.get('name')}")
    print(f"Public repos: {user.get('public_repos')}")
    print(f"Followers: {user.get('followers')}")
# pip install requests
```

Practice tip: Call the Hugging Face model hub API and print the first few results. Handle 429 rate-limit errors.

16. Async with asyncio and httpx

Why this matters for AI

When you call many LLM APIs or download thousands of images, synchronous code is painfully slow. asyncio + httpx lets you run hundreds of network requests concurrently.

Complete Sample

```
# async_demo.py
import asyncio
import httpx

async def fetch_url(client: httpx.AsyncClient, url: str) -> str:
    response = await client.get(url, timeout=10)
    return f"{url} → {response.status_code}"

async def main():
    urls = [
        "https://httpbin.org/delay/1",
        "https://httpbin.org/delay/1",
        "https://httpbin.org/delay/1",
        "https://api.github.com",
    ]
    async with httpx.AsyncClient() as client:
        tasks = [fetch_url(client, url) for url in urls]
        results = await asyncio.gather(*tasks)
    for r in results:
        print(r)

if __name__ == "__main__":
    asyncio.run(main())
# pip install httpx
```

Practice tip: Compare total time of this async version versus a pure requests version downloading the same URLs.

17. Simple CLI with argparse ← NEW

Why this matters for AI

Almost every training or inference script needs command-line arguments (--epochs, --batch-size, --data-path, --lr, etc.). argparse is the standard library solution; typer is a modern alternative if you prefer.

Complete Sample

```
# train_cli.py
import argparse
from pathlib import Path

def main():
    parser = argparse.ArgumentParser(description="Tiny training script example")
    parser.add_argument("--data", type=Path, required=True, help="Path to dataset")
    parser.add_argument("--epochs", type=int, default=10, help="Number of epochs")
    parser.add_argument("--batch-size", type=int, default=32, help="Batch size")
    parser.add_argument("--lr", type=float, default=1e-3, help="Learning rate")
    parser.add_argument("--output", type=Path, default=Path("artifacts"), help="Output dir")
    parser.add_argument("--verbose", action="store_true", help="Print extra logs")
    args = parser.parse_args()

    print("=== Training Config ===")
    print(f"Data path   : {args.data}")
    print(f"Epochs      : {args.epochs}")
    print(f"Batch size   : {args.batch_size}")
    print(f"Learning rate: {args.lr}")
    print(f"Output dir   : {args.output}")
    print(f"Verbose      : {args.verbose}")

    # Pretend training happens here
    if args.verbose:
        print("Starting fake training loop...")
    print("Done.")

if __name__ == "__main__":
    main()

# Run examples:
# python train_cli.py --data ./data/train.csv
# python train_cli.py --data ./data --epochs 50 --batch-size 64 --lr 0.0005 --verbose
```

Practice tip: Add a --seed argument and call the set_seed function from topic 9. Add choices validation for a --optimizer argument (sgd, adam, adamw).

How to Practice Effectively

1. Create folder level1_python_foundations and put every sample in its own file.
2. Run the sample exactly as written. Confirm the output matches the comments.
3. Break it on purpose (wrong types, missing files, bad keys) and watch the errors. Then fix them.
4. Extend each exercise with the "Practice tip" before moving to the next topic.
5. After finishing all topics, build one small combined project that uses at least 8 of them (e.g. a CLI that fetches data from an API, validates it with Pydantic, stores it as JSON, and prints summary statistics).

What Was Added in This Update

- **13. Context managers** — essential for resource management in real pipelines
- **14. collections module** — defaultdict, deque, NamedTuple (daily use in data work)

- **17. argparse CLI** — every training script needs command-line arguments
- pathlib was emphasized inside the File I/O topic

Next Steps

Once comfortable with Level 1, move to **Level 2: Math and Data Tools** (NumPy, Pandas, Matplotlib, basic statistics and linear algebra). That level gives you the numerical foundation required for every machine-learning algorithm that follows.

Hands-On AI with Python • Level 1 Complete Guide
Author: Venkatesan Vetrimurasu | YouTube: Venkat's Tech Lab
Keep coding. Keep breaking things. Keep fixing them.